INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA
SALT LAKE
(Affiliated to UEM Kolkata and Approved by AICTE)

Advanced Programming Lab
[PCCCS495]

University Course Registration System

Name: -Sarbik Chakraborty

Semester: -IV

Class Roll: - 33

Enrolment Number: - 12024052004031

Date: - 29/03/2026

Instructor name: - Dr.Susovan Jana, DR.Soumadip Biswas, Dr.Swagatam Basu

Department of Information Technology

2026

Problem Statement:

Managing course registrations manually can cause many problems in a university setting. This is because traditional methods of course registration lack proper validation and a systematic approach.

The objective of this project is to design a console-based University Course Registration System using Java programming. The system will enable users to perform various functions such as:

Add a new student

Add a course

Add a course to a student

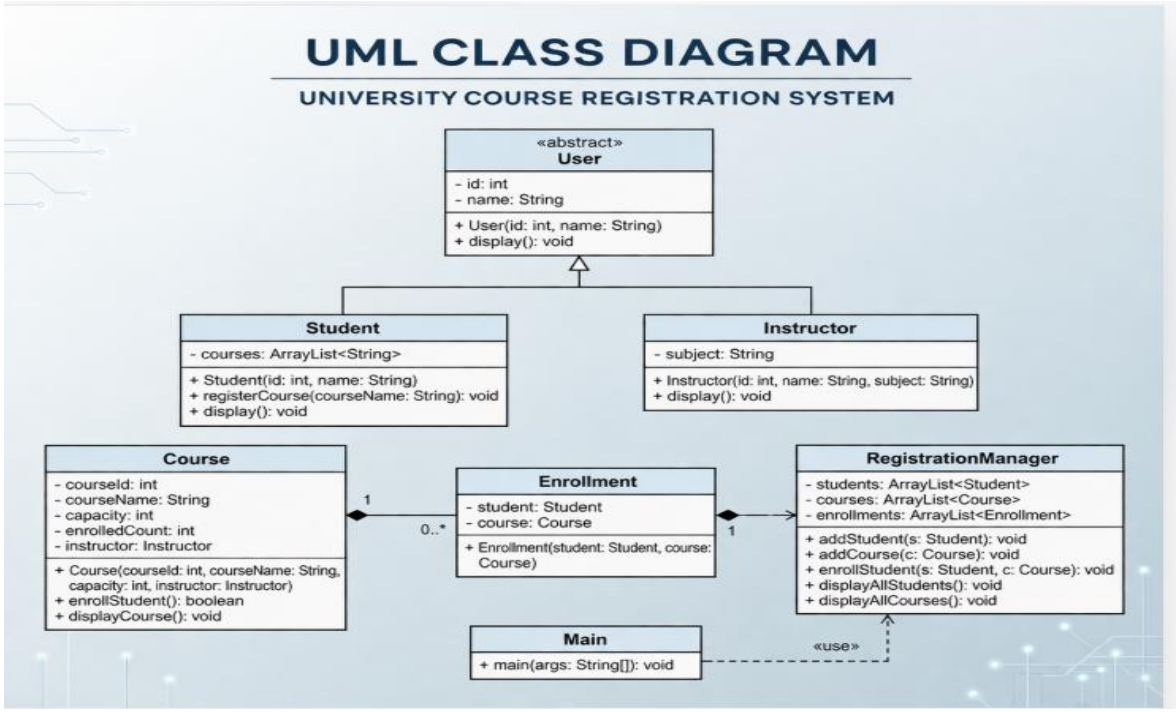
Display a student

Display a course

The system will be designed in a modular and scalable fashion using Object-Oriented Programming concepts.

2. System Design:

2.1 UML Class Diagram (Textual Representation):



2.2 Description of Major Classes:

i. Person (Abstract Class)

- Base class representing common user details

• Contains attributes: id and name
• Defines method display(

ii. Student (Derived Class)

- Extends user
- Stores Registered Courses
- Implements course registration functionality

iii. Instructor(Derieved Class)

- Extends User
- Represents Course Instructor
- Contains Subject Specialization

iv. Course Class, Enrollment Class, RegistrationManager Class and Main Class

- Stores course details and capacity
- Handles enrollment logic
- Tracks no. of students
- Represents relationship between Student and Course.
- Used to manage registration mapping.
- Core controller class
- Manages students,coursed and enrollments
- Handles system operations
- Entry point of program
- Provides menu-driven interface

3.4 Interaction Flow:

1. User starts the program
2. Menu is displayed
3. User selects an option:
    - Add student → stored in list
    - Enroll student → validated and added
    - View Data → display all records
4. System continues until user exits

3. OOP Concepts Demonstrated:

3.1 Abstraction

Where used: Person abstract class

Why used: To define a common structure while hiding implementation details.

Code Snippet:

```
abstract class User {

    int id;

    String name;

}
```

4.1 Abstraction

Where used: User class

```
abstract class User {
    int id;
    String name;
}
```

Explanation:

Defines common structure while hiding implementation.

4.2 Inheritance

Where used: Student & Instructor extend User

```
class Student extends User {
}
```

Explanation:

Reuses common properties and reduces redundancy.

4.3 Polymorphism

Where used: display() method

```
void display() {
    System.out.println(name);
}
```

Explanation:

Different classes can implement method differently.

4.4 Encapsulation

Where used: Class variables

Explanation:

Data is grouped inside classes and accessed through methods.

4.5 Exception Handling

Where used: User input validation

```
try {
    choice = sc.nextInt();
```

```
} catch (Exception e) {
    System.out.println("Invalid input");
}
```

Explanation:
Prevents program crash due to wrong input.

---

5. Implementation Highlights

5.1 Student Addition

```
manager.addStudent(s);
```

Justification:
Stores student data dynamically.

5.2 Course Enrollment Logic

```
if (c.enrollStudent())
```

Justification:
Prevents over-enrollment.

5.3 Data Storage using Collections

```
ArrayList<Student> students;
```

Justification:
Allows dynamic storage and scalability.

5.4 Input Validation

```
try {
    id = sc.nextInt();
}
```

Justification:
Handles invalid inputs safely.

---

6. Testing & Error Handling

6.1 Test Cases

Input	Expected Output
Valid student	Student added
Invalid ID	Error message
Course full	Enrollment denied

6.2 Edge Cases

- No students available
- Invalid numeric input
- Course capacity full
- Wrong menu choice

6.3 Failure Scenarios

• Wrong input → handled by Exception
• Invalid data → prevented
• System crash → avoided

---

7. Git Workflow

• Multiple commits used
• Each class added step-by-step
• Meaningful commit messages maintained
• Development spread across days

---

8. Conclusion & Future Scope

The University Course Registration System successfully demonstrates the application of object-oriented programming principles in a real-world scenario. The system ensures structured design, data validation, and efficient management of course enrollment through a console-based interface.

Future enhancements may include:

• GUI-based interface
• Database integration
• Multi-user support
• Advanced analytics and reporting
.